eks-gitops-platform

Production-grade EKS GitOps platform with SLO-driven reliability

WHAT THIS IS

A reference platform that takes a containerized service all the way from a developer's **git push** to a customer-serving endpoint on AWS - provisioned with Terraform, delivered with GitOps (Argo CD + Helm), guarded by policy-as-code, and operated against a measurable Service Level Objective with automated incident remediation.

Built by **Charan Vamsy** - Senior DevOps / Site Reliability Engineer. This document explains what the project is, how it works, and the evidence that it actually runs.

At a glance	
Repository	github.com/charanvamsy26/eks-gitops-platform (public)
Cloud / runtime	AWS - EKS (Kubernetes 1.30), multi-AZ
Delivery	GitOps via Argo CD (app-of-apps) + Helm
Reliability	99.9% SLO, error-budget burn alerts, auto-remediation
CI/CD	GitHub Actions - test, build, scan, validate (all green)
Files	~220 across IaC, app, charts, policies, docs

1. What the project does

Most "Kubernetes demo" repositories stop at a single `kubectl apply`. This one is built to look and behave like a real platform team's repository: infrastructure is **codified and modular**, deployments are **declarative and auditable**, security is **enforced at multiple layers**, and the running service is **measured against an error budget**. It is both a working reference architecture and portfolio evidence of end-to-end platform ownership.

The system is organised as two cooperating halves: a **delivery** path that turns code into a running service, and an **operate** loop that keeps that service within its reliability target.

2. How it works - the working model

Stage 1 - Delivery (code to running service, via GitOps)

A developer pushes to GitHub. GitHub Actions lints, tests, builds the container image and scans it for vulnerabilities, then publishes it to the GitHub Container Registry. Argo CD detects the change in Git and synchronises the Kubernetes manifests into the EKS cluster. Every deploy is checked at admission by OPA Gatekeeper, which rejects anything that violates policy.

```
Developer  ->  GitHub  ->  GitHub Actions  ->  GHCR  ->  Argo CD  ->  EKS cluster
 writes       git push     test - build -      image     GitOps       runs the
 code         / PR         scan               pushed     sync         demo-api pods
                                                             |
                                             OPA Gatekeeper admits / rejects each deploy
```

Stage 2 - Operate (the SRE reliability loop that defends the SLO)

The demo-api serves traffic and exposes Prometheus metrics. Prometheus scrapes those metrics and continuously evaluates SLO rules against a 99.9% availability target. If errors or latency consume the error budget too quickly, a multi-window burn-rate alert fires. A Python auto-remediation controller reacts - restarting the workload or rolling back via Argo CD - and the service recovers, lowering mean-time-to-recovery.

```
demo-api  ->  Prometheus  ->  error-budget  ->  auto-remediation
 serves +      scrape +        burn alert        restart / rollback
 /metrics      SLO rules       (multi-window)          |
    ^                                                  |
    +---------------- auto-heal -> recovery ----------+
```

3. Components and what each demonstrates

Directory	What it is	Skill shown
terraform/	Modular IaC: VPC, EKS, IAM/IRSA, RDS modules; S3+DynamoDB remote state; dev & prod roots	Terraform, AWS, IaC
app/	demo-api Flask service: /healthz, /readyz, /metrics, fault-injection hooks; non-root image	Python, containers
kubernetes/	Hardened Helm chart (security context, HPA, PDB, NetworkPolicy, ServiceMonitor)	Kubernetes, Helm
argocd/	App-of-apps GitOps: AppProject guardrails + wave-ordered child apps	GitOps, Argo CD
observability/	kube-prometheus-stack values, RED + SLO rules, Grafana dashboards, alerting	Prometheus, Grafana, SRE
policies/	OPA Gatekeeper constraints + Conftest shift-left checks (unit-tested)	DevSecOps, OPA
.github/workflows/	CI/CD: terraform, app, helm, security scanning (tfsec/checkov/trivy/gitleaks)	CI/CD, automation
load-test/ chaos/ tools/	k6 load, chaos injection, and a Python self-healing controller	Reliability engineering

4. The reliability demo (SLO error-budget burn)

The headline feature turns abstract reliability claims into something you can watch. Load is driven at the SLO threshold; chaos is injected to make requests fail; the error budget visibly burns down and an alert fires; the auto-remediation controller detects the sustained burn and restores service. A deliberate design choice: liveness stays healthy during application errors, so pods are not needlessly restarted - readiness and the error budget are what react. That nuance signals senior-level reliability thinking.

5. Is it working? Verification evidence

The service and its reliability logic were run live and exercised end-to-end. The table below lists what was verified and how.

Layer	Verified how	Result
demo-api app + chaos hooks	Ran the live service; exercised endpoints	PASS
Reliability loop	Live: healthy 200 -> chaos -> 500s -> recovered 200	PASS
Metric contract	Live /metrics: http_requests_total{service,path,status}	PASS

Layer	Verified how	Result
Unit tests	pytest: 18 (app) + 42 (controller)	PASS
Helm chart	helm template + kubeconform (9/9 resources valid)	PASS
Policies	opa check + conftest verify (26/26 tests)	PASS
CI/CD pipelines	GitHub Actions: app-ci, security, helm-ci	ALL GREEN
Full cluster (EKS live)	Needs a container runtime; run via 'make demo-up'	READY

Live loop observed: **GET / -> 200** at baseline; after injecting `error_rate=1.0` the same endpoint returned **500** four times while `/healthz` stayed **200**; Prometheus recorded `http_requests_total{...status="500"} 4.0`; after clearing chaos, **GET / -> 200** again.

6. How to run it

Locally on a kind cluster (zero cloud cost) - easiest in GitHub Codespaces where Docker is available:

```
git clone https://github.com/charanvamsy26/eks-gitops-platform && cd eks-gitops-platform
make demo-up          # kind cluster + Prometheus/Grafana + Gatekeeper + demo-api
make demo             # drive the SLO error-budget-burn + auto-remediation demo
make demo-screenshots # capture live Grafana panels into docs/img/
make demo-down        # tear everything down
```

On AWS (real deployment) - provision, then let GitOps take over:

```
cd terraform/environments/dev   # set AWS_PLAN_ROLE_ARN + the documented placeholders
terraform init && terraform apply
kubectl apply -f argocd/bootstrap/root-app.yaml   # Argo CD syncs the rest
```

7. Technology stack

Area	Tools
Cloud & orchestration	AWS, EKS, Kubernetes 1.30, Karpenter, AWS Load Balancer Controller
Infrastructure as code	Terraform, reusable modules, S3 + DynamoDB remote state
Delivery	Argo CD (app-of-apps), Helm, GitHub Container Registry
Observability	Prometheus, Grafana, Alertmanager, multi-window burn-rate SLOs
Security / policy	OPA Gatekeeper, Conftest, tfsec, Checkov, Trivy, gitleaks
Application	Python, Flask, prometheus_client, Docker (non-root, multi-stage)
CI/CD	GitHub Actions, pre-commit, kubeconform, ruff, pytest

Summary. The application, its reliability mechanics, the Helm packaging, the policies, and all CI pipelines are verified working - demonstrated on a running instance, not in theory. The full cluster is one command away. The repository is public and ready to show.